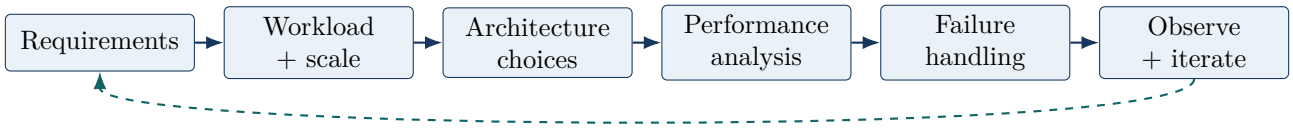
CSD / CSL559

Exhaustive Mid-Sem Study Notebook

System design • Performance • Queueing • Storage • Fault tolerance



How to use this notebook. Read the concept map once, then work through the boxed examples with units visible. Use the question bank without looking at the answer key. For system-design questions, always write: requirements, scale, API/data model, bottleneck, failure path, and trade-offs.

Course anchor. This notebook is based on the supplied System Design Principles, Building Blocks, System Performance, Queueing Theory, RAID, Fault Tolerance, Understanding Distributed Systems, course emails, and sample exam. It follows the instructor's mid-sem flavour: quantitative reasoning plus justified architecture decisions.

Contents

1 The course mental model **1**
1.1 What a large-scale system must do . . . 1
1.2 The seven-step answer pattern . . . 1
1.3 Design principles that explain many answers . . . 1

2 Back-of-the-envelope system sizing **1**
2.1 Core conversions and formulas . . . 2
2.2 Worked example: URL shortener . . . 2
2.3 Common architecture blocks . . . 2
2.4 Partitioning, replication, caching . . . 3
2.5 Mail-thread emphasis: prove the bottleneck first . . . 3

3 Performance and queueing theory **3**
3.1 Notation . . . 3
3.2 Little's Law and M/M/1 . . . 3
3.3 M/M/k, Erlang-B, Erlang-C . . . 4
3.4 Variability and the inspection paradox . . . 4
3.5 P–K formula and resource requirement . . . 4
3.6 Operational laws and closed systems . . . 4

4 Worked sample-paper numericals **5**
4.1 Q3a: Tech Cafe . . . 5
4.2 Q3b: irregular bus arrivals . . . 5
4.3 Q3c: Google Colab staffing . . . 5
4.4 Q3d: Indigo M/M/4 . . . 6
4.5 What each sample-paper step means . . . 6
4.6 Extra worked numericals . . . 6

5 Storage, databases, and distributed data **6**
5.1 RAID: capacity, reliability, performance . . . 7
5.2 LSM trees and WAL . . . 7
5.3 C-Store: WOS and ROS . . . 7
5.4 Dynamo-style stores . . . 7
5.5 SQL vs NoSQL: choose by access pattern . . . 8

6 Fault tolerance, resiliency, and observability **8**
6.1 Fault, error, and failure . . . 8
6.2 Availability and nines . . . 8
6.3 Resiliency patterns . . . 9
6.4 Observability . . . 9

7 System-design case studies **9**
7.1 URL shortener . . . 9

7.2 Newsfeed / Twitter-like timeline . . . 9
7.3 Chat / messaging . . . 10
7.4 File storage / Drive-like system . . . 10
7.5 TrueCaller spam-flag service . . . 10
7.6 Citizen 360 . . . 10
7.7 Fintech loan origination . . . 10
7.8 Illustrated full solutions . . . 11
7.8.1 Drive-like file sync: separate control plane from data plane . . . 11
7.8.2 Notification platform: durable work before unreliable providers . . . 12
7.8.3 Citizen 360: pay the join cost once . . . 12
7.8.4 Resilient dependency call: make overload finite . . . 13
7.9 Sample-paper solution walkthrough . . . 13

8 Question bank: True / False and MCQs 14
8.1 True / False . . . 14
8.2 Why the T/F answers are correct . . . 15
8.3 Multiple-choice questions . . . 15
8.4 MCQ solutions in one line . . . 16

9 Question bank: numericals 16
9.1 Unsolved practice . . . 16
9.2 Numerical answer key . . . 16
9.3 Worked solutions: numerical practice . . . 17

10 Question bank: subjective and design 18
10.1 Short-answer prompts . . . 18
10.2 Long subjective prompts with answer skeletons . . . 19
10.3 System-design prompts . . . 19

11 Two mock exams 20
11.1 Mock Exam A: 90 marks / 90 minutes . . . 20
11.2 Mock Exam A: answer outline . . . 20
11.3 Mock Exam B: 90 marks / 90 minutes . . . 20
11.4 Mock Exam B: answer outline . . . 21

12 Final revision sheets 21
12.1 Formula wall . . . 21
12.2 System-design checklist . . . 21

1 The course mental model

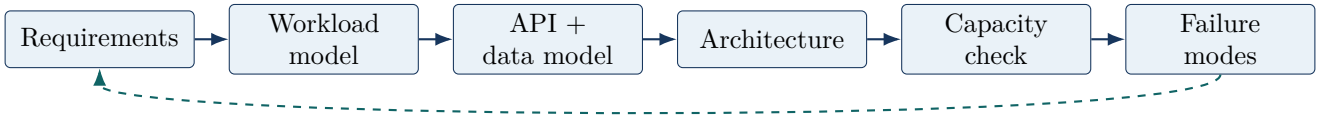
1.1 What a large-scale system must do

A distributed computer system is a collection of components that cooperate over communication links. The course repeatedly returns to six pressures:

Communication	messages can be delayed, reordered, duplicated, dropped, or intercepted.
Coordination	independent nodes must agree enough to make progress, especially during failure.
Scalability	capacity must grow with load without unacceptable latency degradation.
Resiliency	partial failures should not become total outages or corrupt data.
Maintainability	the design must remain understandable, testable, observable, and changeable.
Complexity	interactions, exceptions, and feature additions create emergent behaviour.

1.2 The seven-step answer pattern

1. **Requirements:** what must the system do; what may it relax?
2. **Workload:** users, operations, QPS, payload sizes, read/write ratio, burstiness, hot keys.
3. **Contract:** APIs, entities, IDs, ordering, idempotency, error codes, pagination.
4. **Architecture:** stateless app tier, storage, cache, queue, load balancer, object store, search.
5. **Quantitative check:** capacity, utilization, queueing delay, storage, bandwidth, replication cost.
6. **Failure plan:** timeouts, retries, circuit breakers, failover, consistency, recovery, data loss window.
7. **Operations:** metrics, logs, traces, SLOs, alerts, rollout and rollback.



Exam habit. Never write "use database X" as the design. Write the access pattern first: point lookups, range scans, joins, aggregation, transactions, full-text search, or append-only history. Then justify the technology.

1.3 Design principles that explain many answers

- **Modularity:** isolate responsibilities and contain effects of change/failure.
- **Abstraction:** expose a stable specification, hide implementation details.
- **Layering and hierarchy:** restrict interactions so the number of possible connections does not explode.
- **Naming and indirection:** allow late binding, replacement, routing, and aliases.
- **Robustness:** be tolerant of expected input variation but strict about output/state invariants.
- **Design for iteration:** start with a working small system; measure, learn, and evolve without losing conceptual integrity.
- **KISS:** every "small" feature creates interactions. Simplicity is an explicit design tool.

2 Back-of-the-envelope system sizing

2.1 Core conversions and formulas

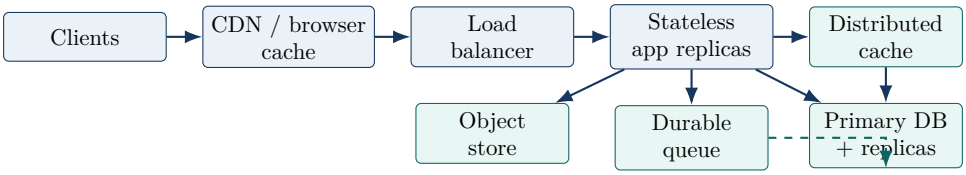
$$\mathrm{QPS}_{avg} = \frac{\text{operations in window}}{\text{seconds in window}}, \qquad \mathrm{QPS}_{peak} = p\,\mathrm{QPS}_{avg}$$
$$\text{storage} = \text{objects} \times \text{bytes/object} \times (1 + \text{overhead}) \times \text{replicas}$$
$$\text{bandwidth} = \text{QPS} \times \text{average payload}, \qquad \text{concurrency} \approx \text{QPS} \times \text{latency}$$

Use 1 day = 86,400 seconds, 1 month $\approx 2.6 \times 10^6$ seconds, 1 year $\approx 3.15 \times 10^7$ seconds, 1 billion $\approx 10^9$, and 1 TB $\approx 10^{12}$ bytes. Keep a peak multiplier and operational headroom; a design at exactly 100% capacity has no room for bursts, failures, or maintenance.

2.2 Worked example: URL shortener

Assumptions. 500 million new URLs per month, read:write = 100 : 1, five-year retention, 500 bytes per mapping, three replicas.
Writes: $500 \times 10^6/(30 \times 86{,}400) \approx 193$ writes/s.
Reads: $193 \times 100 \approx 19{,}300$ reads/s.
Rows: 500M $\times$ 60 = 30 billion.
Raw mapping data: $30 \times 10^9 \times 500 = 15$ TB.
Three replicas: 45 TB before indexes, metadata, compaction, and headroom.
Design consequence: cache hot redirects, use a stateless redirect service, write analytics asynchronously, use random/base62 IDs to avoid enumeration, and paginate analytics by cursor.

2.3 Common architecture blocks



Client/CDN	browser caching, static assets, geo-local delivery.
LB/reverse proxy	health checks, TLS termination, routing, rate limits, backend hiding.
App tier	stateless validation/auth/business logic; scale horizontally.
Cache	low-latency/stale-tolerant reads; TTL/LRU; explicit invalidation.
Database	source of truth; indexes, constraints, transactions, replicas.
Queue	asynchronous work, burst absorption, retry/replay, backpressure.
Object store	large blobs; database stores metadata and object IDs.
Search index	derived full-text/analytics view; not the source of truth.

2.4 Partitioning, replication, caching

Range partitioning	range scans/time windows; newest range can hotspot.	**Hash partitioning**	uniform point lookups; poor range scans.
Consistent hashing	fewer keys move when nodes change; vnodes smooth balance.	**Leader–follower**	simple ordering; leader can be a write bottleneck.
Active–active	scales writes/reads; needs conflict resolution.	**Cache-aside**	read miss fetches DB then fills cache; stale data possible.
Write-through	cache and DB updated together; simpler reads, slower writes.	**CQRS**	separate write model from read/materialized views.

Bottleneck rule: If the database is saturated, adding application servers alone changes little. First reduce database work with indexes, caching, read replicas, partitioning, denormalized read models, or precomputation.

2.5 Mail-thread emphasis: prove the bottleneck first

The instructor's load-balancing assignment is a useful exam pattern: the database is likely to be the bottleneck, the workload is read-heavy, and the design must scale read throughput before adding more front-end machines.

Mini-case. Suppose the application receives 40,000 reads/s and the database safely serves 5,000 reads/s. A cache has hit rate h.

Step 1 — translate the architecture into load:

$$\boxed{\text{DB misses} = (1-h) \times 40{,}000}$$

Step 2 — impose the capacity constraint:

$$\boxed{(1-h)40{,}000 \le 5{,}000 \quad \Rightarrow \quad h \ge 0.875}$$

At $h = 0.90$, the database sees 4,000/s and has headroom. At $h = 0.85$, it sees 6,000/s and is overloaded. The important meaning is that the cache is not merely a speed feature: it changes the work presented to the bottleneck.

How this maps to the mail-thread design: use an in-memory read layer such as Dragonfly/Valkey for hot data, partition the cache by key, and use read replicas or an active–active database when updates and distributed reads must continue across regions. The answer must still state invalidation/freshness, cache-miss behaviour, write ordering, and what happens when the cache is empty. *Technology names earn marks only after the load equation and access pattern are clear.*

3 Performance and queueing theory

3.1 Notation

λ	arrival rate	μ	service rate $= 1/E[S]$
S	service requirement	ρ	utilization; M/M/1 $= \lambda/\mu$
L	mean jobs in system	L_q	mean jobs waiting
W	mean time in system	W_q	mean queue wait
X	throughput	R	resource requirement $= \lambda E[S]$

3.2 Little's Law and M/M/1

$$\boxed{L = \lambda W; \qquad L_q = \lambda W_q}$$

$$\boxed{\rho = \frac{\lambda}{\mu} < 1; \qquad W = \frac{1}{\mu - \lambda} = \frac{E[S]}{1-\rho}; \qquad W_q = \frac{\rho}{\mu - \lambda}}$$

$$L = \frac{\rho}{1-\rho}; \qquad L_q = \frac{\rho^2}{1-\rho}; \qquad P_n = (1-\rho)\rho^n$$

Example. $\lambda = 8$/s and $\mu = 10$/s. Then $\rho = .8$, $W = 1/(10-8) = .5$s, $L = 8(.5) = 4$ jobs, and $W_q = .5 - 1/10 = .4$s. If asked for exactly two customers in the system, use $P_2 = (.2)(.8)^2 = .128$; two waiting would correspond to three in the system, P_3.

3.3 M/M/k, Erlang-B, Erlang-C

Let $a = \lambda/\mu$ be offered load and k the number of identical servers. For M/M/k with a queue:

$$P_0 = \left[\sum_{n=0}^{k-1} \frac{a^n}{n!} + \frac{a^k}{k!}\frac{1}{1-\rho}\right]^{-1}, \qquad \rho = \frac{a}{k}$$

$$P_{wait} = \frac{a^k}{k!}\frac{1}{1-\rho}P_0, \qquad W_q = \frac{P_{wait}}{k\mu - \lambda}$$

For an M/G/k/k loss system with no queue:

$$B(k,a) = \frac{a^k/k!}{\sum_{i=0}^{k} a^i/i!}$$

- P_{wait} means an arrival finds all servers busy and must wait.
- Erlang-B is the probability an arrival is blocked/lost when no waiting room exists.
- A high individual-server utilization does not automatically imply bad delay when there are many servers; the probability all are busy matters.

3.4 Variability and the inspection paradox

$$E[\text{residual wait}] = \frac{E[S^2]}{2E[S]} = \frac{E[S]}{2}(1 + C_S^2), \qquad C_S = \frac{\sigma_S}{E[S]}$$

Bus example. Mean time between buses = 10 min and standard deviation = 30 min. $E[S^2] = \sigma^2 + (E[S])^2 = 30^2 + 10^2 = 1000$. A random observer waits $1000/(2 \cdot 10) = 50$ min on average, not just 5 min.

3.5 P–K formula and resource requirement

For an M/G/1 queue:

$$W_q = \frac{\lambda E[S^2]}{2(1-\rho)}; \qquad W = E[S] + W_q$$

$$R = \lambda E[S]; \qquad k > R \text{ for stability}; \qquad k \approx R + \beta\sqrt{R} \text{ for a service-level target}$$

The square-root rule is a staffing approximation. State the chosen service-level quantile/assumption; do not present it as an exact universal formula.

3.6 Operational laws and closed systems

$$U_i = X_i S_i = X D_i; \qquad X_i = V_i X; \qquad D_i = V_i S_i; \qquad D = \sum_i D_i$$

$$X \le \frac{1}{D_{max}}; \qquad X = \frac{N}{R+Z}; \qquad R = \frac{N}{X} - Z$$

Here V_i is visits per job, S_i service per visit, D_i service demand per job, Z think time, and N the fixed population in a closed system.

4 Worked sample-paper numericals

4.1 Q3a: Tech Cafe

Question. The cafe is free 20% of the time. With Poisson arrivals and exponential service, find the probability that a random observer sees two customers.
Solution. For M/M/1, $P_0 = 1 - \rho$. Hence $1 - \rho = .2$, so $\rho = .8$. Then

$$\boxed{P_2 = (1-\rho)\rho^2 = .2(.8)^2 = .128 = 12.8\%}$$

4.2 Q3b: irregular bus arrivals

Question. Mean time between buses is 10 minutes and standard deviation is 30 minutes. Find the average wait at a random arrival time.
Solution. Use the residual-life/inspection-paradox formula:

$$\boxed{E[W] = \frac{E[S^2]}{2E[S]} = \frac{30^2 + 10^2}{2(10)} = 50 \text{ minutes}}$$

4.3 Q3c: Google Colab staffing

Question. Requests arrive at 10/s and a user is active for 30 minutes. Find resource requirement and rough servers with 200 vCPU each so 90% get one vCPU immediately.
Convert 30 minutes to 1800s:

$$\boxed{R = \lambda E[S] = 10 \cdot 1800 = 18{,}000 \text{ vCPU}}$$

Using square-root staffing with $z_{.90} \approx 1.282$:

$$\boxed{k \approx R + z\sqrt{R} = 18{,}000 + 1.282\sqrt{18{,}000} \approx 18{,}172 \text{ vCPU}}$$

$$\boxed{\text{servers} = \left\lceil \frac{18{,}172}{200} \right\rceil = 91}$$

This is a modelling assumption. If the problem specifies a different service-level model, state it explicitly; the invariant first step is $R = 18{,}000$.

4.4 Q3d: Indigo M/M/4

Question. $\lambda = 2$/min, four agents, each with $\mu = 1$/min. Evaluate the statements.
Offered load $a = \lambda/\mu = 2$ and utilization $\rho = a/k = 2/4 = .5$.

$$P_0 = \left[1 + 2 + \frac{2^2}{2!} + \frac{2^3}{3!} + \frac{2^4}{4!}\frac{1}{1-.5}\right]^{-1} = \frac{1}{7.6667} = .1304,$$

$$P_{wait} = \frac{2^4}{4!}\frac{1}{1-.5}P_0 = 1.3333(.1304) = .1739,$$

$$W_q = \frac{.1739}{4(1)-2} = .08696 \text{ min.}$$

1. "Counter is empty roughly 50% of the time." False: $P_0 = 13.04\%$.
2. "Kiosk probability is higher than waiting probability without kiosk." False. With no queue, $B(4,2) = \frac{2^4/4!}{\sum_{i=0}^{4} 2^i/i!} = .0952$, which is below the no-kiosk wait probability $P_{wait} = .1739$.
3. "Reduce to three agents without much response-time difference." False. For $k = 3$, $\rho = 2/3$, $P_{wait} = 4/9 = .4444$, $W_q = .4444/(3-2) = .4444$ min, about five times the four-agent wait.

4.5 What each sample-paper step means

Tech Cafe	"Free 20%" is P_0, the steady-state probability of zero customers. In M/M/1, that directly reveals utilization: $P_0 = 1 - \rho$.
Bus	A random observer is more likely to arrive inside a long gap than a short one. Therefore the relevant wait is residual life, not half the mean unless intervals are nearly constant.
Colab	$R = \lambda E[S]$ is the mean number of simultaneously occupied vCPUs. The extra $\beta\sqrt{R}$ is a stated stochastic headroom assumption, not a law of nature.
Indigo	P_0 asks whether all agents are idle; P_{wait} asks whether a new arrival sees all agents busy. They are different events. Reducing k raises both utilization and queueing delay nonlinearly.

4.6 Extra worked numericals

1. **M/M/1 delay.** $\lambda = 30$/s, mean service 20ms. Then $\mu = 50$/s, $\rho = .6$, $W = 1/(50-30) = 50$ms, $L = 1.5$ jobs.
2. **P–K.** $\lambda = 4$/s, $E[S] = 0.1$s, $E[S^2] = 0.04\text{s}^2$. $\rho = .4$ and $W_q = 4(.04)/(2(.6)) = .1333$s; $W = .2333$s.
3. **Capacity.** A server handles 250 QPS at 60% target utilization. Peak load 2400 QPS: $\lceil 2400/(250 \cdot .6) \rceil = 16$ servers.
4. **Little's Law.** A service sustains 1200 requests/s with mean latency 80ms. Mean in-flight requests $L = 1200(.08) = 96$.
5. **Availability.** Two independent replicas at $A = .99$: $1 - (.01)^2 = .9999$. If they share a power supply, do not claim independence.

5 Storage, databases, and distributed data

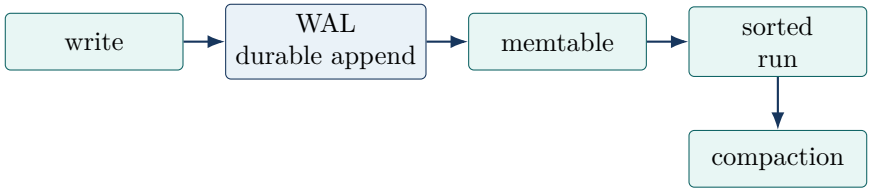
5.1 RAID: capacity, reliability, performance

RAID-0	striping	NB	fast/capacity; any disk failure loses the array.
RAID-1	mirroring	$NB/2$	strong reads/reliability; capacity cost.
RAID-4	dedicated parity	$(N-1)B$	parity disk write bottleneck.
RAID-5	distributed parity	$(N-1)B$	one-disk tolerance; small writes pay read-modify-write.



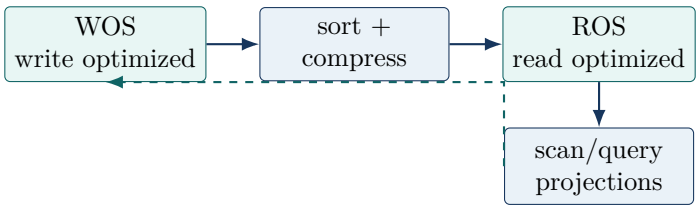
full-stripe writes parallelize; small parity writes pay read–modify–write overhead

5.2 LSM trees and WAL



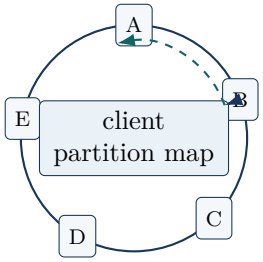
- WAL is written before acknowledgement: after a crash, replay restores the memtable/update intent.
- LSM trees favour sequential writes; trade-offs are read amplification, space amplification, and compaction write amplification.
- Compaction merges immutable sorted runs and removes obsolete versions/tombstones.

5.3 C-Store: WOS and ROS



WOS accepts writes; ROS stores sorted, compressed, read-optimized projections. Move tuples periodically to bound WOS and enable efficient scans. Dictionary, run-length, and delta encoding reduce storage; direct compressed evaluation is possible when predicates operate on encoded values/runs.

5.4 Dynamo-style stores



consistent-hash ring; vnodes split each physical node into smaller ranges

Consistent hashing	key maps to a ring range; fewer keys move when nodes join/leave.
Vnodes	spread load, speed rebalancing, support heterogeneous machines, reduce hotspots; not virtual machines.
Zero-hop DHT	client knows the full partition map and contacts the owner directly.
Quorum	N replicas; R read acknowledgements and W write acknowledgements. Under assumptions, $R + W > N$ creates an intersection.
Sloppy quorum	use healthy substitute nodes when preferred replicas are unavailable.
Hinted handoff	substitute stores a hint and forwards it when the target returns.
Merkle tree	compare range hashes to locate replica divergence efficiently.

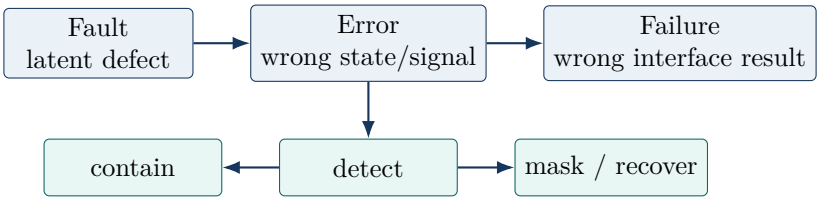
5.5 SQL vs NoSQL: choose by access pattern

	SQL / relational	NoSQL / document / key-value
Schema	strong schema, constraints, joins	flexible schema; application-defined shape
Transactions	rich ACID transactions and integrity	often access-pattern-first; guarantees vary
Queries	ad-hoc joins, aggregation, mature optimizer	predictable key/document queries; denormalization common
Scale	vertical + read replicas/partitioning	horizontal scale often central design goal
Best signal	integrity, joins, multi-row invariants	huge scale, flexible records, predictable access

Sample-paper T/F key: dynamic NoSQL schemas T; SQL data definition/manipulation T; "NoSQL is less cost-effective for big data" F; "NoSQL is usually best for financial integrity" F; "SQL is best when structures change frequently" generally F; Dynamo always uses first preference-list nodes F; LSM append write amplification lower than B-tree T.

6 Fault tolerance, resiliency, and observability

6.1 Fault, error, and failure



- Transient faults may be masked by retry; persistent faults require failover/replacement.
- Modularity is an error-containment boundary. Redundancy enables detection and masking, but correlated redundancy is weak.
- Fail-fast reports invalid state early; fail-stop halts rather than propagating corruption; fail-soft preserves useful partial service.

6.2 Availability and nines

$$A = \frac{\text{uptime}}{\text{uptime+downtime}}$$

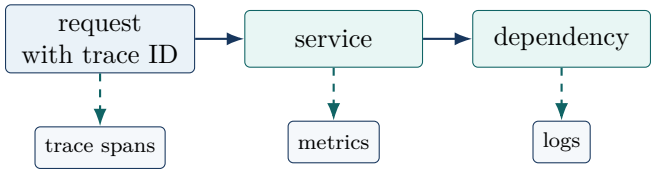
99%	14.4 minutes downtime/day
99.9%	1.44 minutes/day
99.99%	8.64 seconds/day
99.999%	0.864 seconds/day

For independent components, series availability is $\prod_i A_i$ and parallel availability is $1 - \prod_i (1 - A_i)$. State the independence assumption; shared power, rack, region, deployment, or software bug creates correlation.

6.3 Resiliency patterns

Timeout/deadline	bound resource occupancy and prevent a slow dependency from holding every caller.
Retry + back-off/jitter	mask transient faults; retry only safe/idempotent operations and cap attempts.
Circuit breaker	stop sending traffic to a failing dependency; probe before recovery.
Bulkhead	isolate thread pools, queues, tenants, or features so one failure cannot consume all resources.
Rate limiting	protect a service from overload; return controlled rejection.
Load shedding	reject lower-value work to preserve core functionality.
Load leveling	durable queue absorbs bursts; consumers drain at sustainable rate.
Idempotency key	duplicate request/retry produces one logical effect.
Failover/replication	continue after component loss; define consistency and recovery point.

6.4 Observability



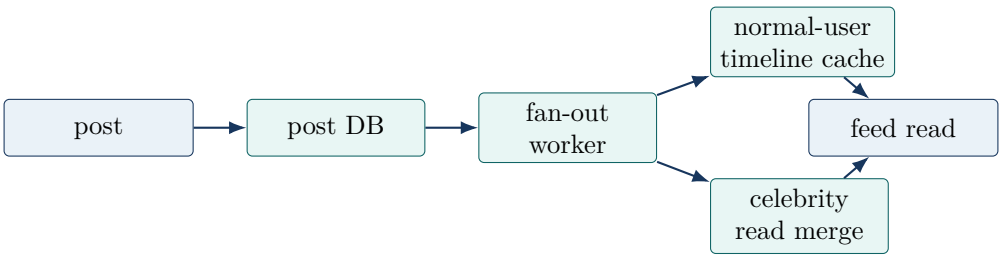
- Metrics: rate, errors, p50/p95/p99 latency, saturation, queue depth, replication lag.
- Logs: structured events, correlation IDs, useful context, privacy-aware fields.
- Traces: request path across services; expose fan-out and slow spans.
- SLI is the measured signal; SLO is the target; alerts should be actionable and tied to error-budget burn.

7 System-design case studies

7.1 URL shortener

Requirements: create short code, redirect quickly, optional expiry/analytics, high availability, unpredictable IDs.
APIs: `POST /shorten`, `GET /{code}`, `DELETE /{code}` or expiry update.
Data: `code -> originalURL, owner, createdAt, expiresAt`; unique code constraint.
Flow: client → CDN/cache → stateless redirect service → replica/read DB; analytics event → queue.
Bottleneck: read QPS and cache miss DB load; shard by code hash.
Failure: cache miss fallback, replica failover, idempotency key on create, async analytics retry.
Trade-off: random IDs improve privacy but require collision detection; write-through cache simplifies reads but makes writes slower.

7.2 Newsfeed / Twitter-like timeline



- Fan-out-on-write is fast for normal users but expensive for celebrities with millions of followers.

- Fan-out-on-read avoids huge writes but makes reads more expensive and potentially less fresh.
- Partition by user ID, salt hot keys, use cursor pagination $(timestamp, postID)$, cache first page, object-store media, and rank asynchronously.

7.3 Chat / messaging

Requirements: ordered messages per conversation, low delivery latency, offline replay, durable history, delivery status.
Data: conversation ID, message ID, sender, sequence number, timestamp, payload, delivery state.
Partition: conversation ID keeps ordering local; a hot group may require subpartitioning with a sequence service.
Path: client → gateway/WebSocket → message service → durable log/DB; fan out to online recipients; offline clients replay from sequence.
Correctness: client-generated idempotency/message ID; deduplicate; ack only after durable append; define at-least-once vs exactly-once effect.

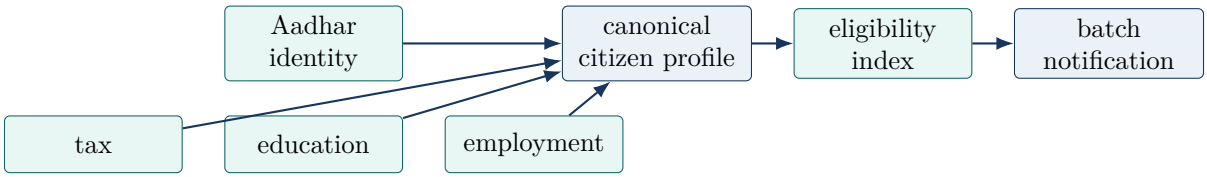
7.4 File storage / Drive-like system

Separate metadata and blobs. Metadata DB stores file tree, owners, ACLs, versions, chunks, hashes; object storage stores immutable chunks.
Upload: negotiate session → upload chunks → checksum/commit manifest → publish version.
Download: metadata lookup → signed object URLs/CDN → parallel chunk fetch.
Scale: content-addressed chunks deduplicate; resumable uploads handle failure; background replication/erasure coding handles durability.
Hard part: concurrent edits need version/conflict policy; ACL checks must apply to metadata and signed URLs.

7.5 TrueCaller spam-flag service

Hot path: phone number → key-value record containing spam score, labels, report count, last updated, and confidence.
Write path: authenticated report → dedupe/rate-limit → durable event → aggregation.
Read path: cache popular numbers; point lookup; return score with freshness.
Abuse controls: reporter reputation, one report per user/number/time window, anomaly detection, appeal/removal path.
Exam phrase: "Use an indexed key-value/inverted lookup; never scan all calls/users for a phone-number query."

7.6 Citizen 360



Naive complexity: scan five $N = 10^9$-row tables per request: $\Theta(5N)$.
Optimized CHECK_ELIGIBILITY: canonical Aadhar key plus indexes/hash maps: $O(k \log N)$ or expected $O(k)$ for k sources; materialized profile is better for repeated reads.
Optimized NOTIFY_SCHEME: refresh eligibility once per annual update; notify K eligible citizens in batches: $O(N)$ refresh or $O(K)$ notification work. Handle missing/obsolete fields, provenance, consent, and idempotent sends.

7.7 Fintech loan origination

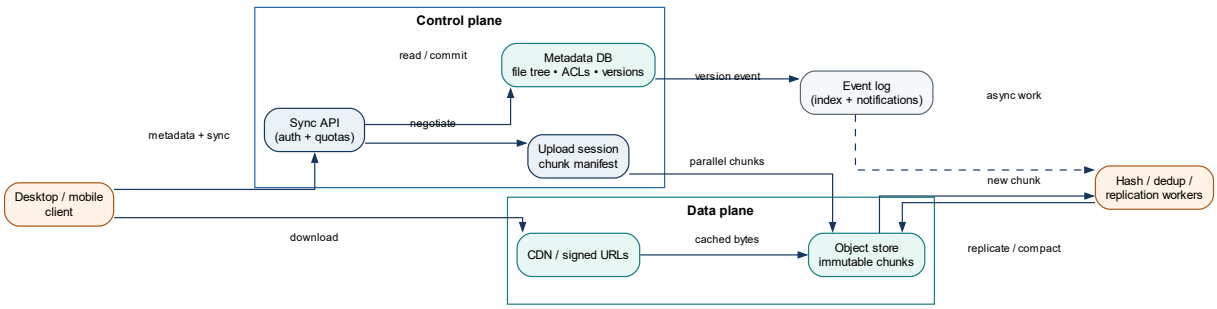
Needs: transactions, auditability, consistent firm-wide view, multiple loan stages/geographies, flexible application data.
Design: transactional source of truth for application state; append-only audit log; read models/materialized

views for dashboards; search index for text/filtering; queue for notifications/document processing.
Choice cue: compared with a pure key-value Dynamo access pattern, a document/relational system is more natural when integrated views, secondary indexes, aggregation, and strong consistency dominate.
Failure: idempotent state transitions, outbox pattern, retryable workers, reconciliation job, immutable audit events.

7.8 Illustrated full solutions

The following four designs are written as model answers. In an exam, the diagram is only the middle of the answer: explain the meaning of each box, the dominant access pattern, the bottleneck, and the failure path.

7.8.1 Drive-like file sync: separate control plane from data plane

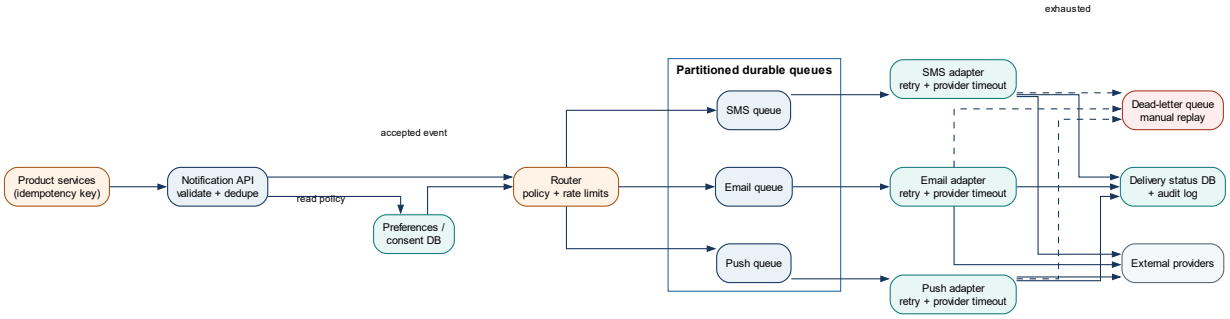


Meaning of the diagram. The *control plane* decides who may access a file, which version is current, and which chunks belong to it. The *data plane* moves large immutable bytes. Keeping them separate prevents a large upload from turning every metadata request into a large database or application-server operation.

1. **Upload:** authenticate; create a resumable session; split the file into chunks; upload chunks directly to object storage; verify hashes; commit one metadata manifest atomically.
2. **Download:** read metadata and ACLs; issue short-lived signed URLs; fetch chunks in parallel through a CDN. The client can retry one missing chunk rather than restarting the whole file.
3. **Correctness:** a version is published only after all referenced chunks pass checksum validation. Concurrent edits create versions; the product chooses last-writer-wins, conflict copies, or merge semantics explicitly.
4. **Failure:** session state survives client restarts; duplicate chunk uploads are harmless through content hashes; a worker retries replication; a committed manifest never points to an unverified chunk.

Main trade-off: content-addressed deduplication saves storage and bandwidth but requires hashing and garbage collection. Metadata is strongly protected; blob replication can be asynchronous when the durability SLO permits it.

7.8.2 Notification platform: durable work before unreliable providers

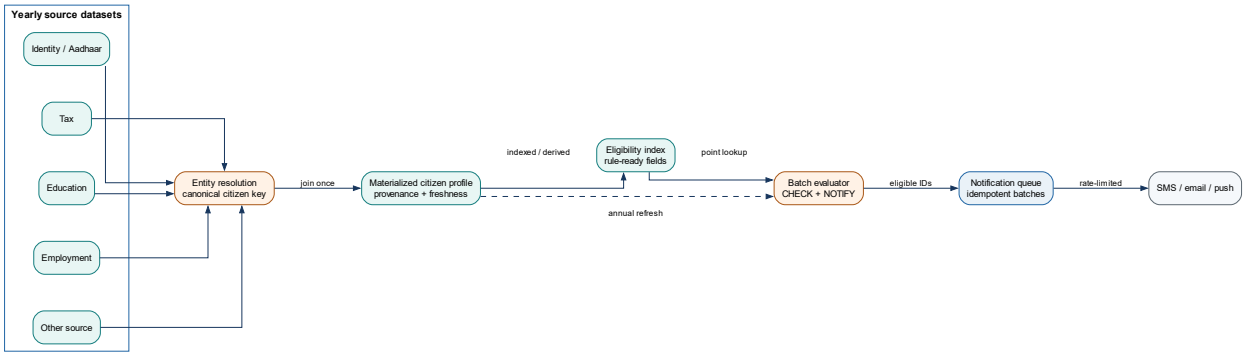


Meaning of the diagram. Product services submit an intent, not a direct provider call. The API validates an idempotency key and stores/accepts the event. Preferences decide whether a channel is allowed. Partitioned queues absorb bursts; channel adapters isolate provider-specific limits and error formats; the status DB records the state seen by users and operators.

1. **Deduplication:** key the accepted event by $(user, template, logical\ event\ ID)$ with an expiry. A repeated producer request returns the original result instead of sending twice.
2. **Retries:** retry only transient errors, with a bounded exponential backoff and jitter. A permanent error or exhausted retry goes to a dead-letter queue for inspection/replay.
3. **Ordering and limits:** partition by recipient or tenant when per-user order matters; apply provider and tenant token buckets; keep large fan-outs in the queue rather than one synchronous request.
4. **Observability:** measure accepted-to-sent latency, provider error rate, queue age, retry count, DLQ size, and delivery success by channel and region.

Main trade-off: accepting an event before delivery gives durability and burst tolerance but makes the API asynchronous. The contract must say that "accepted" is not the same as "delivered."

7.8.3 Citizen 360: pay the join cost once



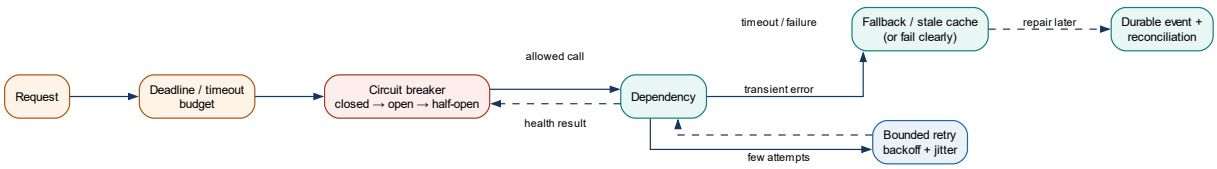
Meaning of the diagram. Five huge yearly source tables are not queried by scanning all rows for every citizen. Entity resolution maps source-specific identifiers to one canonical citizen key. A materialized profile and eligibility index turn repeated checks into indexed point lookups.

1. **Refresh:** validate each source snapshot; normalize identifiers; resolve matches; record provenance and freshness; upsert the canonical profile; build or refresh rule-ready indexes.
2. **CHECK_ELIGIBILITY:** look up the candidate's canonical key, fetch the profile, evaluate the rule, and return the decision plus reason codes and source timestamps. With indexes this is $O(k \log N)$, or expected $O(k)$ with hash lookups, where k is the number of sources/fields touched.

3. **NOTIFY_SCHEME:** evaluate the refreshed profile set once, write eligible IDs to an idempotent queue, and send in batches. The refresh is $O(N)$ in the amount of source data processed; notification work is $O(K)$ for K eligible citizens.
4. **Failure/privacy:** quarantine malformed rows; never silently treat missing data as eligible; retain provenance; encrypt sensitive fields; restrict operators; use an idempotency key so a retry does not double-notify.

Main trade-off: materialization spends storage and refresh time to make online reads predictable. That is the right trade when reads are frequent and source data changes yearly.

7.8.4 Resilient dependency call: make overload finite



Meaning of the diagram. A timeout bounds how long one request can occupy a thread or connection. A circuit breaker stops sending traffic to a dependency that is already failing. Bounded retries are reserved for errors likely to be transient; jitter prevents clients from retrying in lockstep. A fallback preserves a useful degraded result, while reconciliation repairs state later.

1. Set a total deadline first; allocate smaller downstream budgets so the caller does not time out while children continue consuming resources.
2. Retry only idempotent or safely deduplicated operations. Use exponential backoff, jitter, a maximum attempt count, and a retry budget.
3. Open the breaker after an error/latency threshold; probe in half-open; close only after health returns. This is containment, not recovery by itself.
4. Return a clearly labelled stale/partial result when the product permits it. Otherwise fail fast with an actionable error rather than hiding corruption.

7.9 Sample-paper solution walkthrough

Q1 Citizen 360. The naive answer scans 5×10^9 rows per request, so it is $\Theta(5N)$ and cannot scale. The scoring answer identifies a canonical Aadhar key, indexes each source on that key, resolves entities during a yearly refresh, and materializes a profile. Then CHECK_ELIGIBILITY is a small number of indexed reads plus rule evaluation; NOTIFY_SCHEME evaluates the refreshed population once and streams K eligible IDs to an idempotent notification queue. State provenance, missing-field policy, privacy, and the refresh bottleneck.

Q2 storage. Use SQL/transactional storage when financial integrity and multi-row invariants dominate; a key-value lookup is natural for TrueCaller-style phone-number queries; a document or relational read model is natural for a fintech integrated view with secondary filters and aggregation. Explain that zero-hop Dynamo means the client has the complete partition map and contacts the owner directly. Vnodes spread ranges and make rebalancing smoother; hinted handoff stores a temporary replica; Merkle trees locate divergent ranges; WAL makes committed updates replayable; LSM trees exchange compaction cost for sequential, write-friendly ingestion; C-Store moves WOS data into compressed ROS data for analytic scans.

Q3 queueing. Write units first. For M/M/1, infer ρ from P_0 when needed, then use $W = 1/(\mu - \lambda)$ and Little's Law. For the bus, compute $E[S^2] = \sigma^2 + m^2$ before residual wait. For Colab, convert 30 minutes to 1800 seconds and calculate $R = \lambda E[S] = 18{,}000$ vCPUs before stating any staffing approximation. For Indigo, use $a = \lambda/\mu$, $\rho = a/k$, the M/M/k expression for P_0, then P_{wait} and W_q. Always label whether a statement refers to P_0, P_{wait}, or Erlang-B blocking.

8 Question bank: True / False and MCQs

8.1 True / False

Mark T/F and explain in one sentence. The key appears after the questions.

1. In M/M/1, stability requires $\lambda < \mu$.
2. In an M/M/1 queue, $L = \lambda W$ requires Poisson arrivals.
3. If utilization is 0.95, mean delay is always unacceptable regardless of server count.
4. A retry can increase overload during an outage.
5. Consistent hashing guarantees no hotspots.
6. Vnodes are useful for heterogeneous physical nodes.
7. Dynamo is called zero-hop because a client can directly contact the partition owner.
8. Hinted handoff permanently replaces the failed replica.
9. Merkle trees help locate divergent replica ranges.
10. WAL is written after an index update so the index can be reconstructed.
11. LSM trees trade compaction work for write-friendly sequential updates.
12. RAID-0 tolerates one disk failure.
13. Full-stripe parity writes can avoid some read-modify-write work.
14. C-Store WOS is optimized for incoming writes and ROS for compressed reads.
15. A cache is a source of truth by default.
16. Range partitioning is usually good for time-window scans.
17. Hash partitioning is usually good for point lookups.
18. SQL databases cannot support dynamic attributes.
19. NoSQL is automatically the right choice for every big-data workload.
20. Fault, error, and failure are interchangeable terms.
21. Fail-fast means silently returning partial data.
22. Parallel reliability calculations require an independence assumption.
23. A timeout bounds resource occupancy at a dependency.
24. Every retry should be attempted three times.
25. Metrics, logs, and traces answer exactly the same operational question.

T/F key: 1 T; 2 F (Little's Law is much more general); 3 F; 4 T; 5 F; 6 T; 7 T; 8 F; 9 T; 10 F; 11 T; 12 F; 13 T; 14 T; 15 F; 16 T; 17 T; 18 F; 19 F; 20 F; 21 F; 22 T; 23 T; 24 F; 25 F.

8.2 Why the T/F answers are correct

1. $\lambda < \mu$ leaves positive capacity; at equality an M/M/1 queue has no finite steady state.
2. Little's Law is a conservation law and does not require Poisson arrivals.
3. With several servers, one can have high utilization but a small probability that all servers are busy; tail latency and SLO decide acceptability.
4. Retries add load precisely when the dependency is already unhealthy.
5. Consistent hashing limits remapping, but skewed keys and hot objects can still create hotspots.
6. Vnodes divide ownership into many ranges, improving balancing and incremental movement.
7. Zero-hop means the client can compute the owner and route directly, avoiding a coordinating hop.
8. A hinted copy is temporary; when the intended replica returns, the hint is replayed and removed.
9. Merkle-tree hashes let replicas find the first divergent range without comparing every record.
10. WAL must be durable before acknowledging the update; recovery replays it to rebuild state.
11. LSM writes sequentially and postpones organization to compaction, trading write work for read/space amplification.
12. RAID-0 has no redundancy, so one failed disk can lose the entire stripe set.
13. A full-stripe parity write already has all data needed to compute parity, avoiding old-data reads.
14. WOS absorbs writes; ROS stores compressed, read-optimized, mostly immutable columns.
15. A cache is derived state unless the product explicitly makes it authoritative.
16. Range partitions preserve locality for intervals such as time windows.
17. Hash partitions spread point lookups but destroy natural range locality.
18. SQL can use JSON, sparse columns, EAV, or normalized extension tables; dynamic does not mean impossible.
19. Big data is a scale adjective, not a storage decision; access pattern, consistency, and operations still matter.
20. A fault is an underlying defect/event, an error is an incorrect internal state, and a failure is incorrect externally visible behaviour.
21. Fail-fast returns an explicit error quickly; fail-soft returns a deliberately degraded but useful result.
22. Parallel availability calculations multiply failure probabilities only under an independence assumption.
23. Timeouts cap occupancy at one dependency; they do not guarantee the remote operation was cancelled.
24. Retry count depends on idempotency, error class, deadline, and retry budget; three is not universal.
25. Metrics quantify signals, logs preserve event context, and traces connect spans across a request path.

8.3 Multiple-choice questions

1. A system receives 100 requests/s and each request uses 20ms of one CPU. Offered CPU load is: (A) 0.02 (B) 2 (C) 5 (D) 20.
2. In a read-heavy application, the first optimization to investigate is: (A) more UI features (B) cache/read replicas/indexes (C) remove IDs (D) disable monitoring.
3. A replica is down and a substitute stores its update for later forwarding. This is: (A) Merkle tree (B) hinted handoff (C) WAL (D) quorum read.
4. Which mechanism detects divergent replica ranges efficiently? (A) RLE (B) Merkle tree (C) CDN (D) round robin.
5. In M/M/1 with $\lambda = 4$, $\mu = 5$, W is: (A) 0.2 (B) 1 (C) 5 (D) 20 time units.
6. A queue with no waiting room and k servers uses: (A) Erlang-B (B) Little's Law only (C) P–K only (D) SRPT.
7. For a bus with high inter-arrival variance, random waiting is generally: (A) lower than half the mean (B) exactly the mean (C) higher than half the mean (D) zero.
8. A good partition key should primarily: (A) be constant (B) distribute load for the dominant access pattern (C) maximize joins (D) be secret from the application.
9. In C-Store, tuples move from: (A) ROS to WOS only (B) WOS to ROS (C) WAL to CDN (D) cache to DNS.

10. A circuit breaker primarily: (A) compresses logs (B) stops calls to a failing dependency (C) creates IDs (D) increases retry rate.

MCQ key: 1 B; 2 B; 3 B; 4 B; 5 B; 6 A; 7 C; 8 B; 9 B; 10 B.

8.4 MCQ solutions in one line

1. **B:** $100 \times 0.020 = 2$ CPU-seconds/s of offered load.
2. **B:** read replicas, indexes, and cache reduce work at a read bottleneck.
3. **B:** a substitute stores a hinted copy for a failed replica.
4. **B:** Merkle trees compare hashes hierarchically.
5. **B:** $W = 1/(5-4) = 1$ time unit.
6. **A:** Erlang-B models loss when all k servers are busy and there is no queue.
7. **C:** variability creates long gaps and length-biased observation, increasing residual wait.
8. **B:** the key must distribute the dominant workload and avoid hot partitions.
9. **B:** incoming tuples are first buffered in WOS, then reorganized into ROS.
10. **B:** a circuit breaker prevents calls from amplifying a dependency failure.

9 Question bank: numericals

9.1 Unsolved practice

1. An M/M/1 service receives 6 jobs/s and serves 10 jobs/s. Find ρ, L, L_q, W, and W_q.
2. A service receives 50 requests/s. Mean service time is 12ms. Find R and minimum stable server count.
3. A service receives 500 QPS, each response averages 40KB. Find average bandwidth in MB/s and approximate in-flight requests at 80ms latency.
4. A bus has mean interval 8 min and standard deviation 12 min. Find random-observer wait.
5. M/M/3: $\lambda = 4$/min and $\mu = 2$/min/agent. Find offered load, utilization, P_0, P_{wait}.
6. M/M/4/4 with offered load $a = 3$. Find Erlang-B blocking probability.
7. An application writes 2M objects/day of 2KB for 3 years, with 3 replicas and 25% overhead. Estimate storage.
8. A closed system has $N = 20$, think time $Z = 5$s, measured throughput $X = 2$/s. Find response time.
9. A two-device system has demands $D_1 = 0.04$s and $D_2 = 0.10$s. Find bottleneck throughput bound.
10. An M/G/1 has $\lambda = 5$/s, $E[S] = 0.1$s, $E[S^2] = 0.03\text{s}^2$. Find ρ, W_q, and W.
11. A 99.9% component is duplicated independently in parallel. Find combined availability and downtime per day.
12. A cache receives 20,000 reads/s with 92% hit rate. DB capacity is 500 reads/s. Is the DB safe? Include the miss load.

9.2 Numerical answer key

1. $\rho = .6$, $L = 1.5$, $L_q = .9$, $W = .25$s, $W_q = .15$s.
2. $R = 50(.012) = .6$ server-seconds/s; minimum stable count $k = 1$ (target utilization/headroom may require more).
3. 500×40KB $= 20{,}000$KB/s ≈ 20MB/s; in-flight $= 500(.08) = 40$.
4. $(12^2 + 8^2)/(2 \cdot 8) = 13$ min.

5. $a = 2$, $\rho = 2/3$; $P_0 = [1 + 2 + 2 + 4]^{-1} = 1/9$; $P_{wait} = 4/9$.
6. $B(4, 3) = \frac{3^4/4!}{1+3+4.5+4.5+3.375} \approx .2061$.
7. $2\text{M} \times 365 \times 3 \times 2\text{KB} \times 3$ replicas $\times 1.25 \approx 16.43$ TB.
8. $R = N/X - Z = 20/2 - 5 = 5$s.
9. $X \leq 1/.10 = 10$ jobs/s.
10. $\rho = .5$; $W_q = 5(.03)/(2(.5)) = .15$s; $W = .25$s.
11. $1 - (.001)^2 = .999999$, about 0.0864 seconds/day.
12. Miss load = 20,000(.08) = 1600/s; DB is not safe at 500/s without more replicas/partitioning/precomputation.

9.3 Worked solutions: numerical practice

1. M/M/1. $\lambda = 6$/s, $\mu = 10$/s. The utilization is $\rho = 6/10 = .6$. Then

$$\boxed{W = \frac{1}{10 - 6} = .25 \text{ s}, \quad L = 6(.25) = 1.5, \quad W_q = W - 1/10 = .15 \text{ s}, \quad L_q = 6(.15) = .9.}$$

Here W includes service time; W_q is only the line, and Little's Law converts time into average jobs.

2. Resource requirement. 50/s with mean service 12ms = .012s gives

$$\boxed{R = \lambda E[S] = 50(.012) = .6.}$$

This is 0.6 continuously occupied server equivalents. One server is mathematically stable, but a real target utilization below 100% requires headroom.

3. Bandwidth and concurrency. 500 QPS $\times$40KB = 20,000KB/s $\approx$ 20MB/s. At 80ms = .08s latency,

$$\boxed{L = \lambda W = 500(.08) = 40 \text{ in-flight requests.}}$$

The bandwidth is a traffic estimate; it is not automatically the required number of servers.

4. Residual bus wait. With mean $m = 8$ min and $\sigma = 12$ min,

$$\boxed{E[S^2] = \sigma^2 + m^2 = 144 + 64 = 208, \qquad E[W] = \frac{208}{2(8)} = 13 \text{ min.}}$$

The answer exceeds $m/2 = 4$ min because high variance makes long gaps more likely to contain the observer.

5. M/M/3. Here $a = \lambda/\mu = 4/2 = 2$ and $\rho = a/k = 2/3$. The normalization terms are $1 + 2 + 2$ for states $0, 1, 2$, plus $a^3/(3!(1 - \rho)) = 4$ for the queue tail. Thus

$$\boxed{P_0 = 1/9, \qquad P_{wait} = 4P_0 = 4/9 \approx .444.}$$

6. Erlang-B. With $k = 4$ and offered load $a = 3$,

$$\boxed{B(4, 3) = \frac{3^4/4!}{1 + 3 + 3^2/2! + 3^3/3! + 3^4/4!} = \frac{3.375}{16.375} \approx .2061.}$$

About 20.6% of arrivals are blocked when there is no waiting room.

7. Storage. Three years contain $2\text{M} \times 365 \times 3 = 2.19\text{B}$ objects. Logical bytes are $2.19\text{B} \times 2\text{KB} = 4.38\text{BKB}$. Add three replicas and 25% overhead:

$$\boxed{4.38 \times 3 \times 1.25 = 16.425\text{B KB} \approx 16.43 \text{ TB}.}$$

State whether decimal or binary units are being used; the scale and replica factor are the important marks.

8. Closed system. Rearrange $X = N/(R+Z)$:

$$\boxed{R = \frac{N}{X} - Z = \frac{20}{2} - 5 = 5 \text{ s}.}$$

This is the mean time spent receiving service across the system, excluding the 5-second think time.

9. Bottleneck bound. For demands $D_1 = .04$s and $D_2 = .10$s, the second device is the bottleneck. Therefore

$$\boxed{X \le \frac{1}{D_{max}} = \frac{1}{.10} = 10 \text{ jobs/s}.}$$

This is a bound, not a promise: queueing, synchronization, and other work can make the achieved throughput lower.

10. M/G/1. $\rho = \lambda E[S] = 5(.1) = .5$. Then

$$\boxed{W_q = \frac{5(.03)}{2(1-.5)} = .15 \text{ s}, \qquad W = .1 + .15 = .25 \text{ s}.}$$

The second moment $E[S^2]$, not just the mean, captures the delay penalty of service variability.

11. Parallel availability. Two independent $A = .999$ replicas fail together with probability $(.001)^2$:

$$\boxed{A_{parallel} = 1 - (.001)^2 = .999999.}$$

The downtime fraction is 10^{-6}, so per day it is $86{,}400 \times 10^{-6} = .0864$ seconds. Shared power or a common network invalidates the independence claim.

12. Cache safety. The miss rate is $1 - .92 = .08$, so the database receives

$$\boxed{20{,}000(.08) = 1{,}600 \text{ reads/s}.}$$

That exceeds its 500 reads/s capacity. The minimum hit rate for safety is $1 - 500/20{,}000 = .975$, or 97.5%, before extra headroom.

10 Question bank: subjective and design

10.1 Short-answer prompts

1. Explain why the "second-system effect" is dangerous.
2. Compare range and hash partitioning with one workload for each.
3. Why can adding a cache create correctness problems?
4. Differentiate fail-fast, fail-stop, and fail-soft.

5. Why are retries with no jitter dangerous?
6. Explain read amplification, write amplification, and space amplification in LSM trees.
7. What do WOS and ROS do in C-Store?
8. Why are vnodes valuable in Dynamo?
9. Explain how a WAL supports crash recovery.
10. Why can two replicas still fail together?
11. What is the difference between an SLI and an SLO?
12. Why should large blobs usually be stored outside a relational row?
13. Explain why "more servers" may not reduce response time.
14. Compare fan-out-on-write and fan-out-on-read for newsfeeds.
15. What makes a request handler idempotent?

10.2 Long subjective prompts with answer skeletons

Q. Design a citizen eligibility service over five 1-billion-row databases.
Answer: Canonical identity; indexed Aadhar resolution; materialized profile; annual refresh; $O(k \log N)$/expected $O(k)$ eligibility lookup; $O(N)$ refresh or $O(K)$ notification; batching, idempotency, missing-field policy, provenance, privacy, and bottleneck analysis.

Q. Compare DynamoDB and MongoDB for a firm-wide loan dashboard.
Answer: Start from access patterns. Dynamo-style key-value scale is strong for predictable point access and availability; an integrated dashboard needing secondary indexes, aggregation, flexible documents, transactions/consistency, and multiple views favours MongoDB or a relational system. State trade-offs rather than claiming one is universally better.

Q. Explain how to make a multi-service payment workflow reliable.
Answer: Transactional source of truth, idempotency key, outbox event, durable queue, idempotent consumers, timeouts/backoff, circuit breaker, reconciliation, immutable audit log, strong state transitions, metrics/traces, and manual recovery path.

Q. Why can a centralized queue outperform independent queues?
Answer: A central queue sends work to any free server, preventing one idle server while another has work. It improves utilization but introduces dispatcher/queue scalability, fault, fairness, and scheduling concerns.

10.3 System-design prompts

1. Design a URL shortener with analytics and expiry.
2. Design a Twitter-like newsfeed with celebrity accounts.
3. Design a chat system with offline delivery and per-conversation ordering.
4. Design a Drive-like file sync system with versioning and conflict handling.
5. Design a video upload/transcoding platform.
6. Design a nearby restaurant/proximity search service.
7. Design a ticket-booking system with scarce inventory and no overselling.
8. Design a notification platform for SMS, email, and push.
9. Design a rate limiter for a public API.
10. Design a metrics/log ingestion pipeline.

11. Design a fraud/spam detection lookup service.
12. Design a distributed job scheduler with retries and priorities.

For every design prompt, answer in this order: requirements → scale → APIs/entities → high-level flow → data/partitioning → cache/queue/replication → bottleneck/capacity → failure/consistency → metrics/SLOs → trade-offs.

11 Two mock exams

11.1 Mock Exam A: 90 marks / 90 minutes

Instructions: closed book; state assumptions and units. Show justification for True/False and design choices.

Q1. Public benefits eligibility platform [30 marks]

Five yearly-updated datasets contain 1 billion rows each: identity, tax, education enrollment/pass, and employment. Design:
(a) **CHECK_ELIGIBILITY [10]** for a candidate with identity, education, income, and employment fields. Give entity-resolution strategy and running time.
(b) **NOTIFY_SCHEME [10]** to notify all eligible candidates. Give running time and optimization.
(c) **Architecture [10]** including refresh pipeline, indexes/materialized views, batch notifications, failures, and privacy.

Q2. Distributed storage [30 marks]

(a) Mark T/F and justify: dynamic schemas; SQL integrity; vnodes; hinted handoff; Merkle trees; WAL; RAID-5 small-write penalty. [10]
(b) Explain how C-Store loads WOS, moves tuples to ROS, and answers compressed queries. [8]
(c) Choose SQL, document, or key-value storage for (i) financial ledger, (ii) spam-number lookup, (iii) flexible loan application dashboard. [6]
(d) Explain one consistency trade-off and one failure-recovery mechanism. [6]

Q3. Queueing and capacity [30 marks]

(a) M/M/1 with $\lambda = 6$/s, $\mu = 10$/s: find ρ, L, L_q, W, W_q. [8]
(b) A bus has mean interval 12 min and standard deviation 18 min. Find random wait. [5]
(c) A service receives 25/s and uses 40ms average service time. Find R and rough servers for 80% utilization. [6]
(d) Four agents have $\lambda = 3$/min and $\mu = 1$/min. Find probability the system is empty and probability an arrival waits; comment on reducing to three agents. [11]

11.2 Mock Exam A: answer outline

Q1: canonical key + indexed/materialized profile; point lookups $O(k \log N)$ or expected $O(k)$; notification refresh $O(N)$ and send $O(K)$; idempotent batch queue.
Q2: SQL/transactional for ledger; key-value for phone lookup; document/relational for flexible integrated dashboard; WOS→ROS; WAL before ack; RAID-5 parity/small writes.
Q3(a): $\rho = .6$, $L = 1.5$, $L_q = .9$, $W = .25$s, $W_q = .15$s.
Q3(b): $(18^2 + 12^2)/(2 \cdot 12) = 15$ min.
Q3(c): $R = 25(.04) = 1$; $k \approx 1/.8 = 1.25 \Rightarrow 2$ rough servers.
Q3(d): $a = 3$, $k = 4$, $\rho = .75$; $P_0 = [1 + 3 + 4.5 + 4.5 + 13.5]^{-1} = 1/26.5$; $P_{wait} = 13.5/26.5 \approx .509$; reducing to 3 makes $\rho = 1$ unstable.

11.3 Mock Exam B: 90 marks / 90 minutes

Q1. Application architecture [30 marks]

Design a notification service that sends SMS/email/push for millions of users. Include APIs, deduplication, preferences, rate limits, queueing, provider failures, retries, delivery status, partition key, and observability.

Q2. Concepts [30 marks]

(a) Explain consistent hashing, vnodes, quorum, sloppy quorum, hinted handoff, and Merkle trees. [10]
(b) Compare cache-aside, write-through, and materialized views. [6]
(c) Explain fault/error/failure, detection/containment/masking, and fail-soft design. [8]
(d) Give four reasons that "just retry" can make an outage worse. [6]

Q3. Numericals [30 marks]

(a) A cache sees 12,000 QPS with 95% hit rate; the DB handles 400 QPS. Is it safe? What hit rate is needed? [6]
(b) M/G/1: $\lambda = 3/\text{s}$, $E[S] = 0.2\text{s}$, $E[S^2] = 0.08\text{s}^2$. Find ρ, W_q, W. [8]
(c) A closed system has $N = 50$, $Z = 10\text{s}$, throughput $X = 2/\text{s}$. Find response time. [5]
(d) Three independent replicas have availability .98 each. Find combined availability and downtime/day. [5]
(e) A 1000 QPS service has 50ms mean latency. Find in-flight requests and explain why this is not a capacity calculation by itself. [6]

11.4 Mock Exam B: answer outline

(a) Miss load $= 12{,}000(.05) = 600/\text{s} > 400$, unsafe. Needed hit rate: $1 - 400/12{,}000 = .9667$, or 96.67%.
(b) $\rho = .6$; $W_q = 3(.08)/(2(.4)) = .3\text{s}$; $W = .5\text{s}$.
(c) $R = N/X - Z = 50/2 - 10 = 15\text{s}$.
(d) $1 - (.02)^3 = .999992$, downtime $\approx 0.6912\text{s/day}$.
(e) $L = 1000(.05) = 50$ in-flight; capacity also needs service time distribution, concurrency limits, CPU/IO demand, and target tail latency.

12 Final revision sheets

12.1 Formula wall

$$
\begin{aligned}
\text{Little:} &\quad L = \lambda W, \quad L_q = \lambda W_q \\
\text{M/M/1:} &\quad \rho = \lambda/\mu, \quad W = 1/(\mu - \lambda), \quad L = \rho/(1-\rho) \\
\text{M/M/1 queue:} &\quad W_q = \rho/(\mu - \lambda), \quad L_q = \rho^2/(1-\rho), \quad P_n = (1-\rho)\rho^n \\
\text{M/G/1:} &\quad W_q = \lambda E[S^2]/(2(1-\rho)), \quad W = E[S] + W_q \\
\text{Resource:} &\quad R = \lambda E[S], \quad k > R, \quad k \approx R + \beta\sqrt{R} \\
\text{Residual:} &\quad E[W] = E[S^2]/(2E[S]) \\
\text{Closed:} &\quad X = N/(R+Z), \quad R = N/X - Z \\
\text{Bottleneck:} &\quad X \le 1/D_{max} \\
\text{Availability:} &\quad A_{series} = \prod A_i, \quad A_{parallel} = 1 - \prod(1 - A_i)
\end{aligned}
$$

12.2 System-design checklist

1. What are the functional requirements and the one most important non-functional requirement?
2. What is the average/peak workload, read/write ratio, payload, retention, and hot-key pattern?
3. What are the API contracts and stable identifiers?
4. What is the source of truth? What is derived/cache/search data?

5. What is the partition key and what happens to hot partitions?
6. Where are replicas, queues, caches, and object storage used?
7. What resource is the bottleneck? Show one capacity equation.
8. What happens when each dependency is slow, unavailable, duplicated, or inconsistent?
9. Which operations are idempotent? What is the retry/reconciliation story?
10. What metrics, logs, traces, SLOs, and alerts prove that the system works? Name one trade-off and one evolution path.